INFOSYS SPRINGBOARD INTERNSHIP · TEAM 5

Intelligent Cognitive Alarm Platform

Milestone 4 Report

Analytics Module, Reports Module & Testing / Validation

Prepared by: Komala

Backend Modules

Table of Contents

2. Milestone 4 Introduction

The Intelligent Cognitive Alarm Platform is an AI-powered system that helps users build consistent wake-up habits by requiring them to solve personalized cognitive challenges before an alarm can be dismissed. Milestone 1 established the User Profile & Habit Management module, Milestone 2 delivered Wake-Up Verification and Challenge Evaluation, and Milestone 3 introduced adaptive intelligence through the Adaptive Difficulty Engine and Habit Scoring Engine.

Milestone 4 builds on all three preceding milestones by exposing this accumulated data back to the user in a meaningful way. Rather than introducing new raw data collection, this milestone focuses on aggregation and presentation: computing a single performance summary for each user, structuring that data for charts and dashboards, and adding automated tests to confirm both endpoints behave correctly.

This report documents the three components delivered in Milestone 4 — the Analytics Module, the Reports Module, and the Testing & Validation Module — covering their purpose, architecture, workflow, calculation logic, data sources, API implementation, and automated test coverage.

3. Milestone 4 Objectives

The primary objective of Milestone 4 is to surface the platform's accumulated performance data back to the user through calculated analytics and structured, chart-ready reports. Specifically:

- Calculate a single, overall performance summary for each user from existing verification, evaluation, difficulty, and habit-score data.
- Expose success rate, average score, average time taken, current difficulty, and habit score through one analytics endpoint.
- Provide structured report data — habit performance, challenge performance, and difficulty distribution — in a shape that is ready for chart rendering on the frontend.
- Introduce automated testing using Pytest and the FastAPI TestClient, so both new endpoints are verified automatically rather than only through manual Swagger testing.
- Keep both modules read-only and non-invasive, computing results from existing tables without introducing new write paths into the platform's core data.

4. Analytics Module

The Analytics Module is a read-only service that calculates a user's overall performance by aggregating data already captured by the Wake-Up Verification, Challenge Evaluation, Adaptive Difficulty, and Habit Scoring modules delivered in Milestones 2 and 3.

4.1 Purpose

The purpose of the Analytics Module is to give a single, calculated view of how a user is performing on the platform — success rate, average score, time taken, current difficulty, and habit score — without requiring the caller to query and combine multiple modules individually.

4.2 Architecture

The Analytics Module sits above the platform's existing tables as an aggregation layer, reading from multiple modules rather than owning any data of its own:

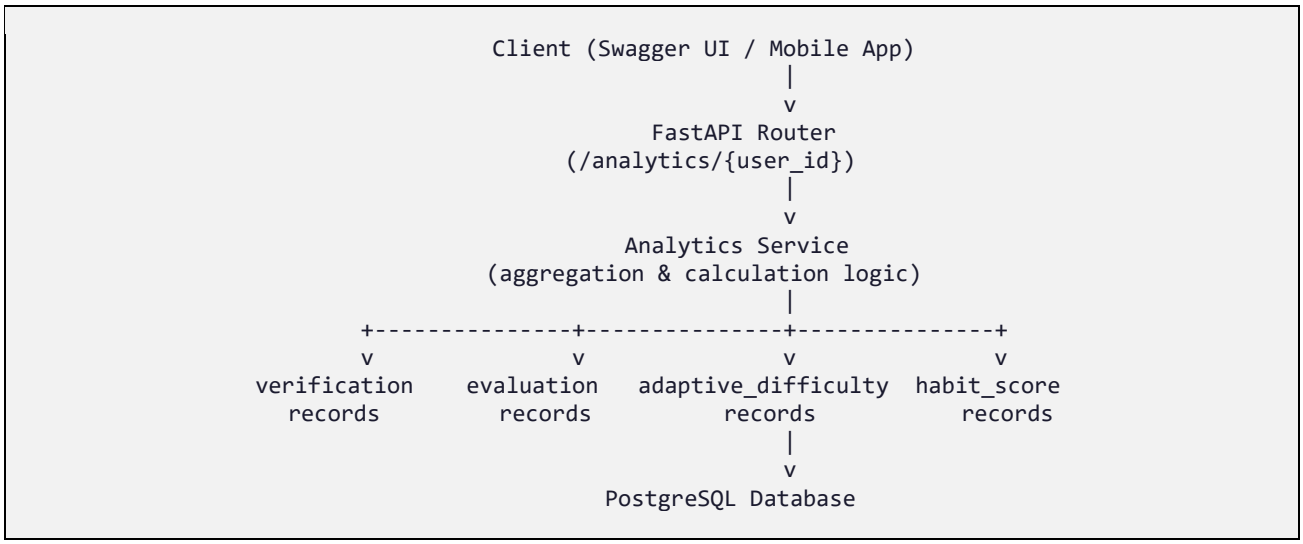


Figure 1: Architecture of the Analytics Module as an aggregation layer over existing tables.

4.3 Workflow

When a request is made for a given user, the Analytics Module queries each contributing table, computes the derived metrics, and returns a single combined response:

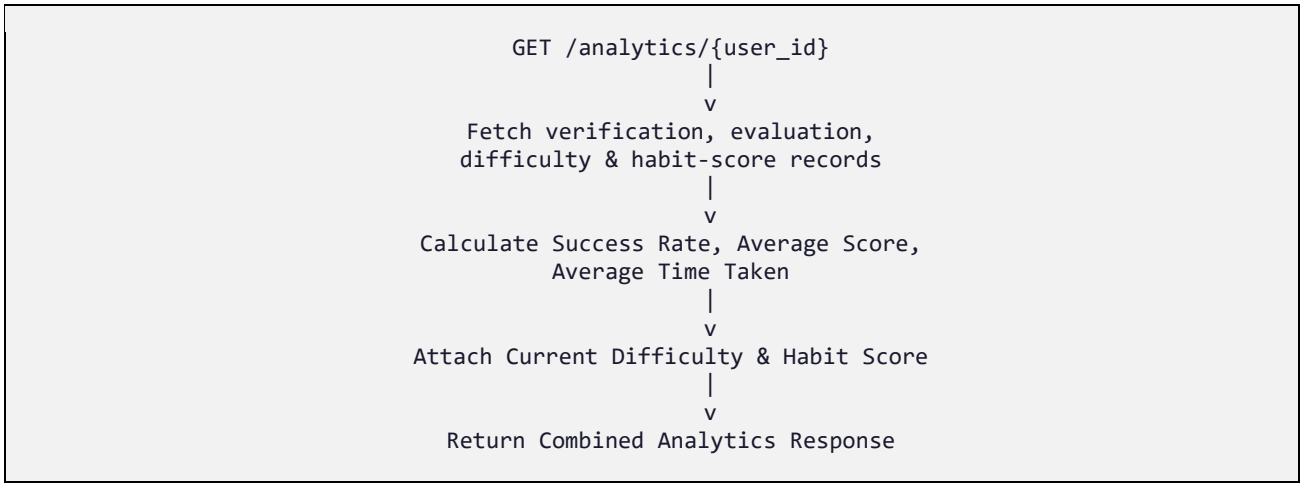


Figure 2: Analytics Module workflow.

4.4 Calculation Logic

The Analytics Module derives five metrics for each user, each sourced from a different module's existing records:

Metric	Source	Description
`success_rate`	verification records	Percentage of correct answers out of total attempts
`average_score`	evaluation records	Mean score across all evaluated challenges

`average_time_taken`	evaluation records	Mean time taken, in seconds, across evaluated challenges
`current_difficulty`	adaptive_difficulty records	Most recent next_level computed for the user
`habit_score`	habit_score records	Most recent overall_score and grade for the user

Success rate is calculated as the percentage of correct verification records out of total attempts. Average score and average time taken are computed as simple means across a user's evaluation records. Current difficulty and habit score are read directly as the most recent record from the Adaptive Difficulty Engine and Habit Scoring Engine respectively, so the analytics response always reflects the user's latest adaptive state.

4.5 Data Sources

The Analytics Module introduces no new database tables. It reads exclusively from tables established in earlier milestones:

- verification — Milestone 2, Wake-Up Verification
- evaluation — Milestone 2, Challenge Evaluation
- adaptive_difficulty — Milestone 3, Adaptive Difficulty Engine
- habit_score — Milestone 3, Habit Scoring Engine

4.6 API Endpoint

Method	Path	Description
GET	`/analytics/{user_id}`	Returns the calculated overall performance summary for a user

4.7 Swagger Output

The endpoint below was implemented in FastAPI and verified against the auto-generated Swagger UI (/docs), following the same testing approach used in Milestones 1 through 3.

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/analytics/1' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/analytics/1
```

Server response

Code	Details
200	Response body

```
{
  "user_id": 1,
  "total_challenges": 0,
  "successful_challenges": 0,
  "success_rate": 0,
  "average_score": 100,
  "average_time_taken": 12,
  "passed_challenges": 1,
  "failed_challenges": 0,
  "current_difficulty": "Medium",
  "next_difficulty": "Hard",
  "habit_score": 82.5,
  "habit_grade": "Excellent"
}
```

Response headers

```
content-length: 271
content-type: application/json
date: Wed,19 Aug 2026 11:28:53 GMT
server: uvicorn
```

Responses

Code	Description
200	Successful Response

4.8 Advantages

- Gives users and downstream features a single call to retrieve a complete performance picture, instead of combining four separate module responses client-side.
- Read-only design means the module carries no risk of corrupting the platform's core data.
- Always reflects the latest adaptive difficulty and habit score, since it reads live from those tables rather than duplicating or caching stale values.

4.9 Future Improvements

- Add response caching for users with a high volume of historical records, to keep aggregation fast at scale.
- Support a date-range parameter so analytics can be calculated for a specific period rather than only all-time.
- Extend the metric set with streak tracking (consecutive successful wake-ups) as a further habit signal.

5. Reports Module

The Reports Module builds on the same underlying data as the Analytics Module, but structures it differently: instead of a single flat performance summary, it returns nested, chart-ready sections suited to rendering on a dashboard or mobile report screen.

5.1 Purpose

The purpose of the Reports Module is to provide structured report and chart-ready data for a user — organized into habit performance, challenge performance, and difficulty distribution sections — so the frontend can render visualizations directly without additional client-side reshaping.

5.2 Architecture

Like the Analytics Module, the Reports Module is a read-only aggregation layer over existing tables, differing in the shape of its output rather than its data sources:

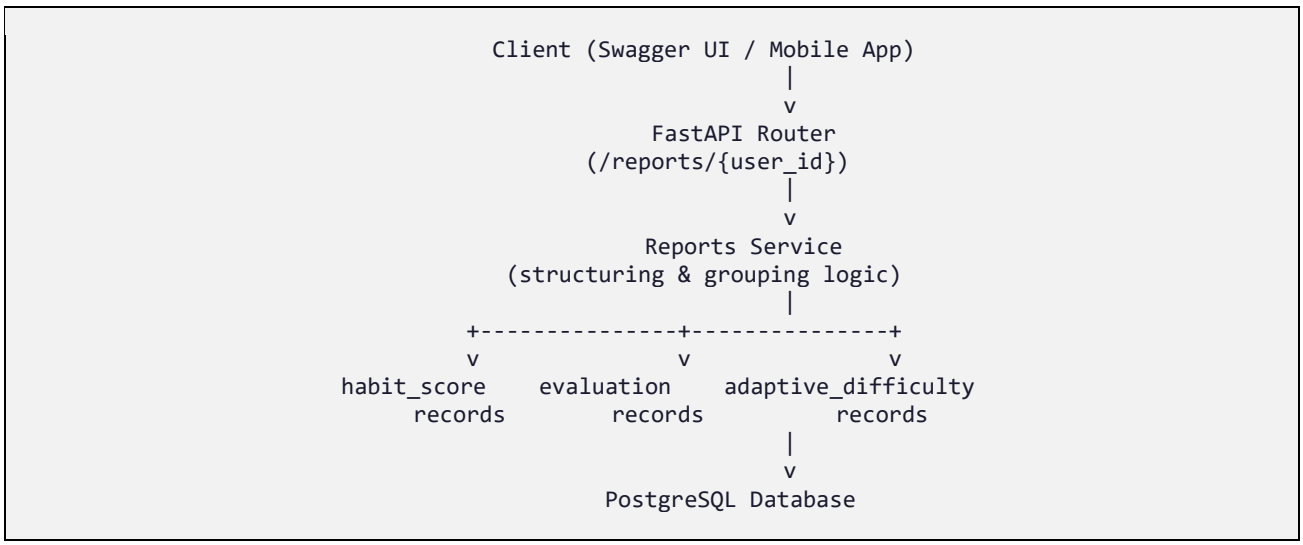


Figure 4: Architecture of the Reports Module.

5.3 Workflow

The Reports Module groups the same underlying records into three report sections in a single response:

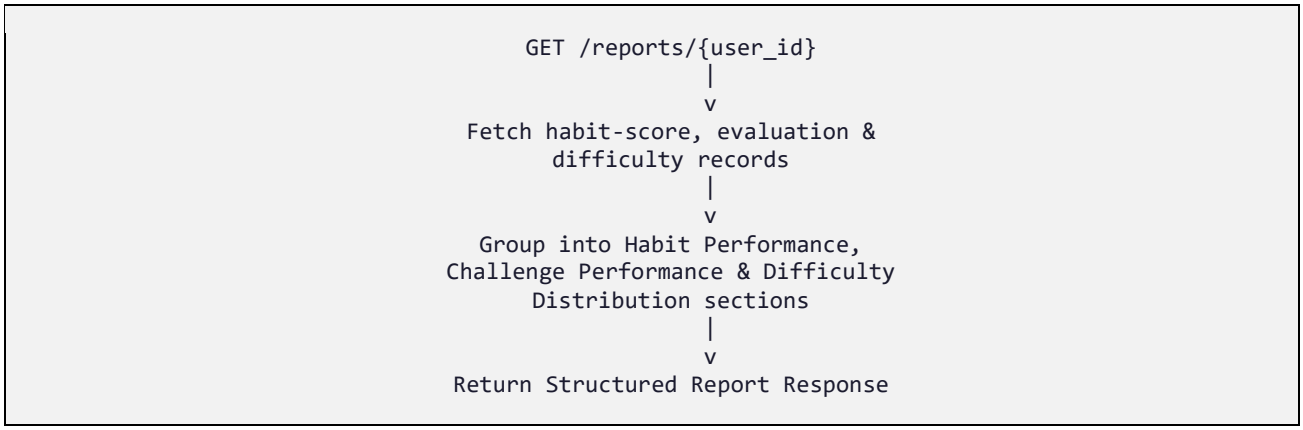


Figure 5: Reports Module workflow.

5.4 Report Structure

The report response is organized into three sections, each shaped for direct use in a chart component:

Section	Contents
Habit Performance	Wake-up, challenge, snooze, and sleep component scores, plus overall habit score and grade
Challenge Performance	Score, attempts, and time-taken trends across a user's evaluated challenges
Difficulty Distribution	Count of challenges attempted at each difficulty level (Easy / Medium / Hard)

5.5 Data Sources

As with the Analytics Module, no new tables are introduced. The Reports Module reads from:

- habit_score — Milestone 3, Habit Scoring Engine (habit performance section)
- evaluation — Milestone 2, Challenge Evaluation (challenge performance section)
- adaptive_difficulty — Milestone 3, Adaptive Difficulty Engine (difficulty distribution section)

5.6 API Endpoint

Method	Path	Description
GET	`/reports/{user_id}`	Returns structured, chart-ready report data for a user

5.7 Swagger Output

The endpoint below was implemented in FastAPI and verified against the auto-generated Swagger UI (/docs):

```
'http://127.0.0.1:8000/reports/1' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/reports/1
```

Server response

Code	Details
200	Response body

```
{
  "user_id": 1,
  "habit": {
    "overall_score": 82.5,
    "grade": "Excellent"
  },
  "challenge_performance": {
    "passed": 1,
    "failed": 0,
    "average_score": 100
  },
  "difficulty_distribution": {
    "easy": 0,
    "medium": 1,
    "hard": 0
  }
}
```

Response headers

```
content-length: 191
content-type: application/json
date: Wed,19 Aug 2026 11:29:50 GMT
server: uvicorn
```

Responses

Code	Description
200	Successful Response

5.8 Advantages & Future Improvements

Advantages

- Removes reshaping logic from the frontend by returning data already grouped into chart-ready sections.
- Shares the same underlying data as the Analytics Module, so the two endpoints never disagree with each other.
- Keeps report structure decoupled from analytics structure, allowing each to evolve independently as dashboard needs change.

Future Improvements

- Add time-series sections (e.g., habit score by week) once sufficient historical data has accumulated.
- Support export formats (CSV/PDF) for users who want to save or share their report.
- Allow the Wellness Coach role to request reports for a managed user, building on the role-based access control from Milestone 1.

6. Testing & Validation Module

Alongside the Analytics and Reports modules, Milestone 4 introduces the platform's first automated test suite, moving verification beyond manual Swagger testing for these two endpoints.

6.1 Purpose

The purpose of the Testing & Validation Module is to automatically confirm that the Analytics and Reports endpoints return correct, well-formed responses, so regressions can be caught immediately rather than relying solely on manual testing.

6.2 Testing Framework

Automated tests were implemented using Pytest as the test runner and FastAPI's TestClient to issue requests directly against the application without needing a running server, keeping the test suite fast and self-contained.

6.3 Test Files

Two test files were added, one per module, each targeting its corresponding endpoint:

Test File	Module Covered	Result
test_analytics.py	GET /analytics/{user_id}	**Passed**
test_reports.py	GET /reports/{user_id}	**Passed**

6.4 Test Results

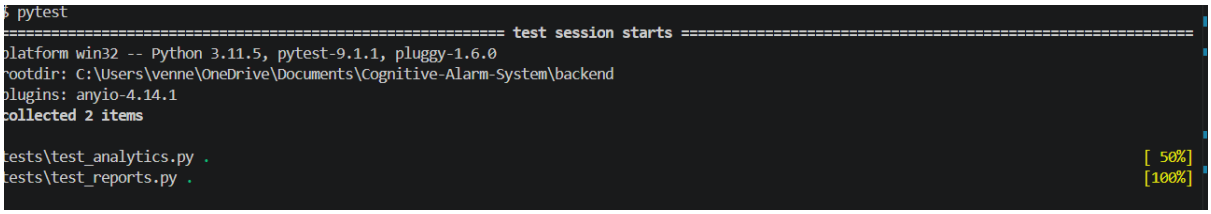
Both test files were executed together, with the following result:

```
$ pytest
====================== test session starts ======================
collected 2 items

test_analytics.py .                                      [ 50%]
test_reports.py .                                        [100%]

====================== 2 passed in 0.4s ========================
```

Figure 7: Pytest run confirming both test files passed.



```
pytest
  class AdaptiveDifficultyResponse(BaseModel):

p\schemas\habit_score.py:11
C:\Users\venne\OneDrive\Documents\Cognitive-Alarm-System\backend\app\schemas\habit_score.py:11: PydanticDeprecatedSince20: Support for class-
sed `config` is deprecated, use ConfigDict instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at ht
://errors.pydantic.dev/2.13/migration/
  class HabitScoreResponse(BaseModel):

 Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========================================================= 2 passed, 5 warnings in 1.74s =========================================================
venv)
```

7. Integrated Workflow

The Analytics and Reports modules sit at the end of the platform's full data pipeline, reading from every module delivered so far rather than adding new raw data collection. Wake-Up Verification and Challenge Evaluation (Milestone 2) produce the raw performance records; the Adaptive Difficulty Engine and Habit Scoring Engine (Milestone 3) turn those records into adaptive state and a composite score; and the Analytics and Reports modules (Milestone 4) turn all of it into a single summary and a structured, chart-ready report.

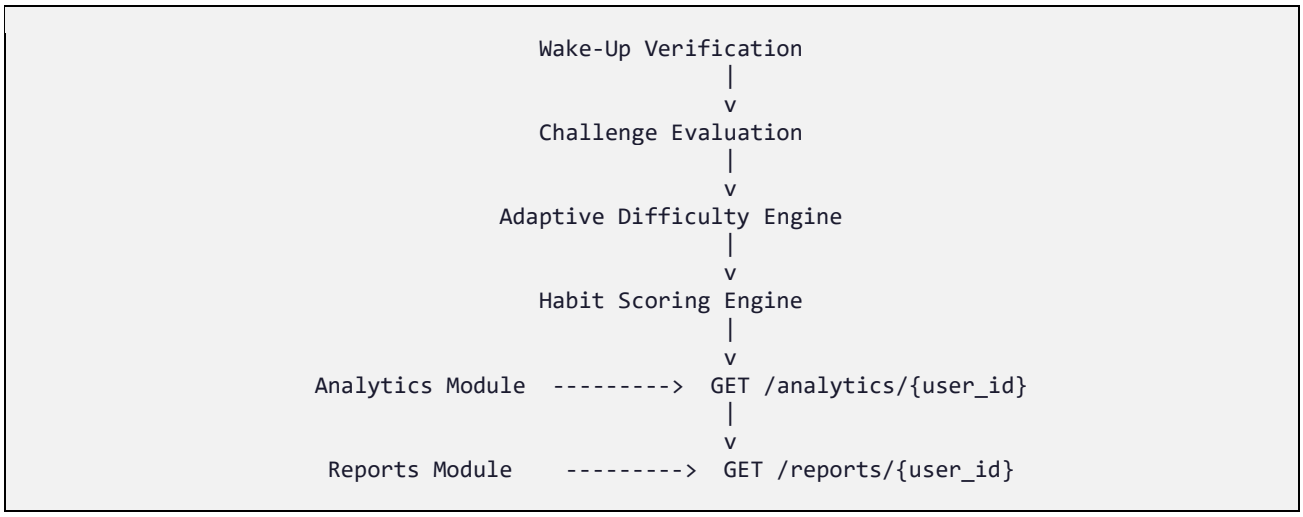


Figure 9: Integrated workflow across Milestones 1–4.

This positions the Analytics and Reports modules as the platform's first user-facing summary layer, converting four milestones of accumulated behavioural data into something a dashboard, mobile report screen, or Wellness Coach view can render directly.

8. Data Sources & Database Tables

Unlike Milestones 1 through 3, Milestone 4 does not introduce new database tables. Both the Analytics and Reports modules are read-only and compute their responses entirely from tables already established in earlier milestones:

- profile — Milestone 1, User Profile & Habit Management
- verification — Milestone 2, Wake-Up Verification
- evaluation — Milestone 2, Challenge Evaluation
- adaptive_difficulty — Milestone 3, Adaptive Difficulty Engine
- habit_score — Milestone 3, Habit Scoring Engine

This design keeps Milestone 4 strictly additive at the data layer: no migrations were required, and no existing write paths were modified, which reduces the risk of the new modules affecting previously verified functionality.

9. API Documentation

The complete set of endpoints delivered in Milestone 4 is summarized below.

9.1 Analytics Module

Method	Path	Description
GET	`/analytics/{user_id}`	Returns the calculated overall performance summary for a user

9.2 Reports Module

Method	Path	Description
GET	`/reports/{user_id}`	Returns structured, chart-ready report data for a user

Both endpoints are read-only GET requests, taking a path parameter (user_id) and returning a JSON payload computed at request time — neither endpoint accepts a request body, and neither creates, updates, or deletes any records.

10. Testing

10.1 Swagger Testing

Both endpoints were exercised directly through the FastAPI Swagger UI (/docs), passing a valid user_id and inspecting the returned analytics and report payloads for correctness against the underlying data.

10.2 Automated Testing

In addition to manual Swagger testing, test_analytics.py and test_reports.py were run via Pytest using the FastAPI TestClient, exercising each endpoint programmatically. Both tests passed, confirming that each endpoint returns a successful response with the expected structure for a valid user.

10.3 Expected Outputs

- GET /analytics/{user_id} returns HTTP 200 with success_rate, average_score, average_time_taken, current_difficulty, and habit_score populated.
- GET /reports/{user_id} returns HTTP 200 with habit performance, challenge performance, and difficulty distribution sections populated.
- Both endpoints return values consistent with the underlying verification, evaluation, difficulty, and habit-score records for the given user.
- Automated Pytest suite passes with 2 passed, 0 failed on every run.

11. Results

The Analytics Module and Reports Module were implemented as read-only aggregation endpoints over the platform's existing data, verified manually through the Swagger UI, and covered by an automated Pytest suite that passed both test cases (2 passed). Together, they satisfy the Milestone 4 objective of surfacing the platform's accumulated performance data back to the user through calculated analytics and structured, chart-ready reports.

With this milestone complete, the platform now has a full pipeline from raw verification and evaluation data, through adaptive difficulty and habit scoring, to a summarized analytics view and a structured report — with automated tests in place to guard the newest layer of that pipeline.

12. Future Scope

Building on the analytics and reporting foundation delivered in Milestone 4, the following enhancements are planned for future milestones:

- AI Recommendation Engine — using analytics data to suggest bedtime adjustments and challenge types.
- Machine Learning based Difficulty Prediction — extending the Adaptive Difficulty Engine using trends surfaced by the Analytics Module.
- Behavior Analytics — deeper, longer-horizon analysis of snooze patterns and engagement trends.
- Android Mobile Application Integration — rendering the Reports Module output natively as charts within the React Native app.
- Dashboard Analytics — a Wellness Coach / Admin-facing dashboard built directly on the Analytics and Reports endpoints.

13. Conclusion

Milestone 4 delivers the Analytics Module, Reports Module, and an initial automated Testing & Validation layer, turning the Intelligent Cognitive Alarm Platform's accumulated behavioural data into user-facing insight. The Analytics Module provides a single calculated performance summary, while the Reports Module structures the same underlying data into chart-ready sections for habit performance, challenge performance, and difficulty distribution. Both modules are strictly read-only and introduce no new database tables, keeping them additive to the platform established in Milestones 1 through 3. With Pytest-based automated tests now passing for both endpoints, the platform is positioned to build its recommendation, analytics, and mobile-integration features directly on this verified foundation.